

던전 시스템 기획서

작성자 : 하광선

목차



1 기획 의도

1. 던전 시스템 : 기획 의도
2. 던전 제작 방식 표준화
3. 개발 효율성 향상

2 설계

1. 던전 시스템 구성
2. 입장 판단
3. 던전 세팅
4. 조건 판정
5. 이벤트 실행
6. 던전 종료 판정
7. 결과 & 보상 처리

3 데이터 및 변수

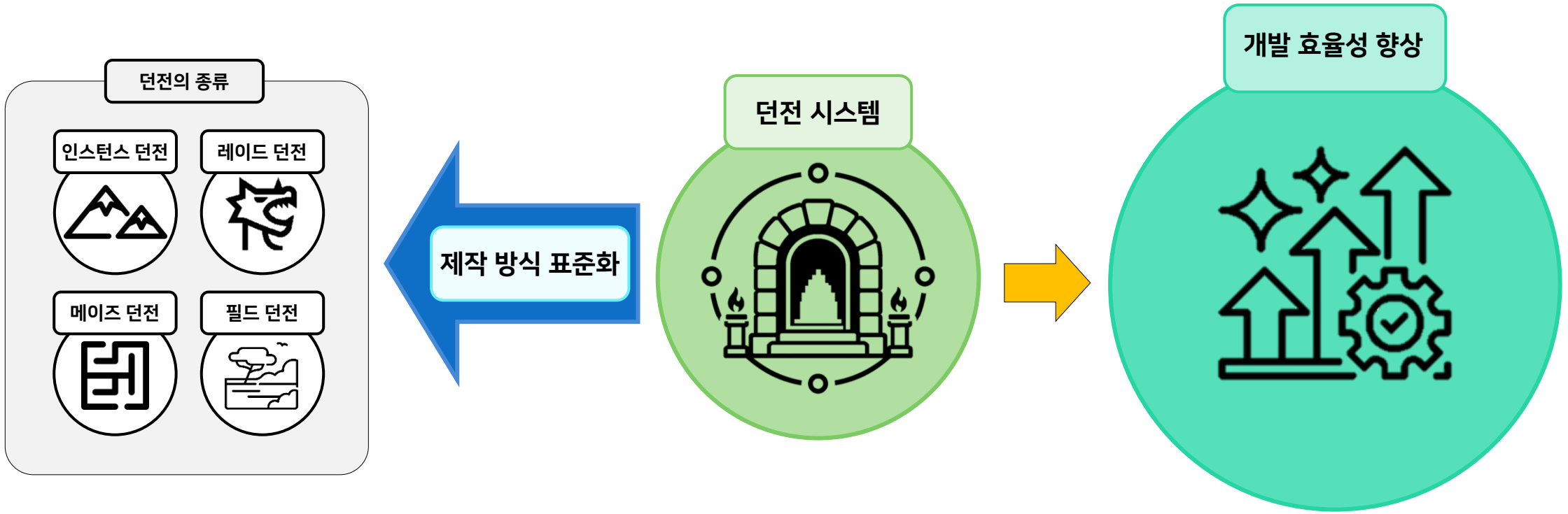
1. 테이블 개요
2. 테이블 범례



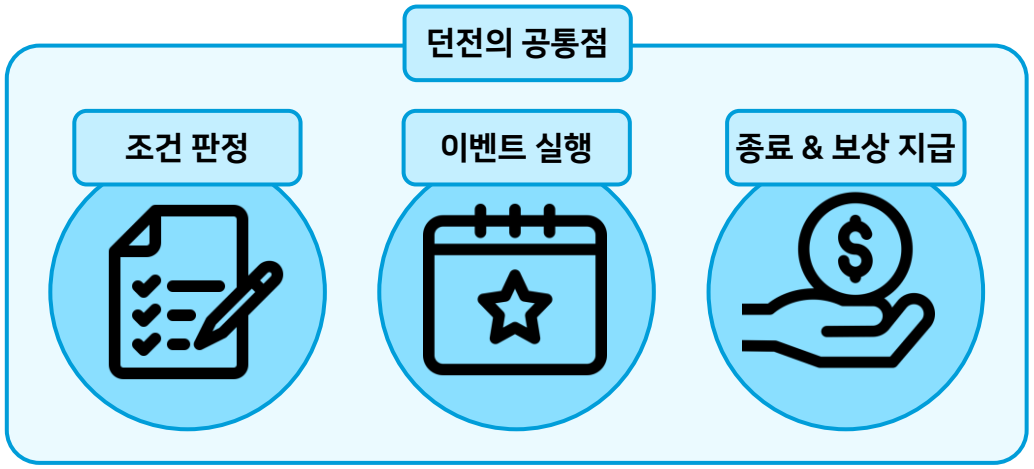
Ⅰ. 기획 의도

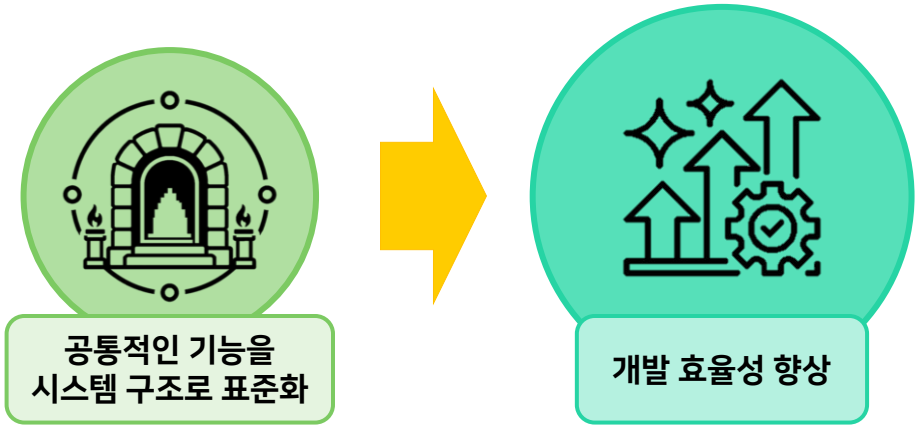
던전 시스템 : 기획 의도

다양한 형태의 던전 콘텐츠를 공통 시스템 구조로 관리하여 던전 제작 방식을 표준화하고,
신규 콘텐츠 제작과 기존 콘텐츠 유지 보수 등의 개발을 효율적으로 수행할 수 있도록 한다.



던전 제작 방식 표준화





1 MMORPG 던전의 다양한 종류

MMORPG의 던전은 레이드, 메이즈, 단일 맵 던전 등 다양한 형태로 구성된다.
각 던전은 진행 방식과 콘텐츠 구성에 차이가 있지만, 플레이어의 입장 이후 진행 조건을 달성하고 결과를 판정하는 공통적인 흐름을 가진다.
따라서 던전의 종류와 관계없이 동일한 시스템 구조를 적용할 수 있으며, 이를 통해 다양한 콘텐츠를 일관된 방식으로 관리할 수 있다.

2 던전의 공통점 & 표준화의 장점

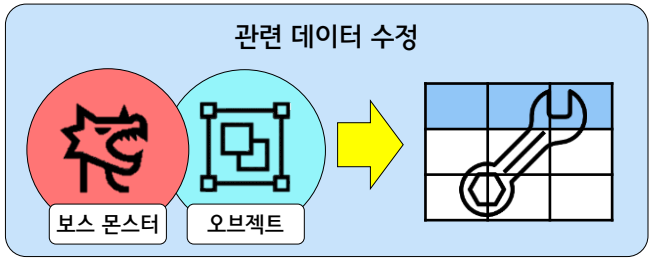
던전은 진행 조건 달성, 이벤트 발생, 종료 조건 판정 등 공통적인 기능을 반복적으로 사용한다.
이러한 공통 요소를 하나의 시스템 구조로 표준화하면 던전마다 별도의 로직을 새로 구현할 필요가 없으며, 동일한 기준으로 콘텐츠를 제작할 수 있다.
또한 개발자와 기획자가 동일한 제작 규칙을 공유하게 되어 콘텐츠 품질을 일정하게 유지하고 관리 효율성을 향상시킬 수 있다.

개발 효율성 향상



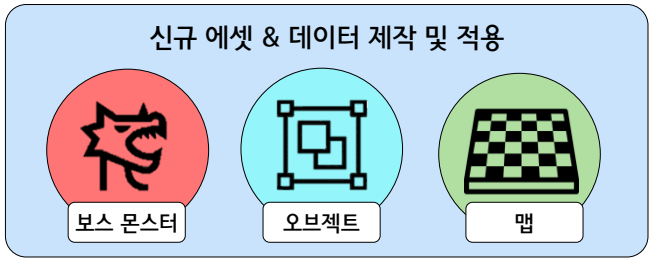
목표 : 기존 던전의 수정 버전 제작

기존 콘텐츠의 난이도를 낮춰야 한다.



목표 : 신규 던전 제작

새로운 레이드를 제작하려 한다.



1 표준화가 어떻게 개발 효율성으로 이어지는가

공통 시스템 구조를 기반으로 던전을 제작할 경우, 신규 콘텐츠 개발 시 기존 구조와 기능을 재사용할 수 있다.
개발자는 던전별 전용 로직을 구현하는 대신 몬스터, 조건, 이벤트, 보상 등의 데이터만 추가하여 콘텐츠를 구성할 수 있으며, 이를 통해 개발 기간과 구현 비용을 절감할 수 있다.
또한 동일한 구조를 사용하기 때문에 QA 과정에서도 테스트 범위를 예측하기 쉬워 개발 효율성이 향상된다.

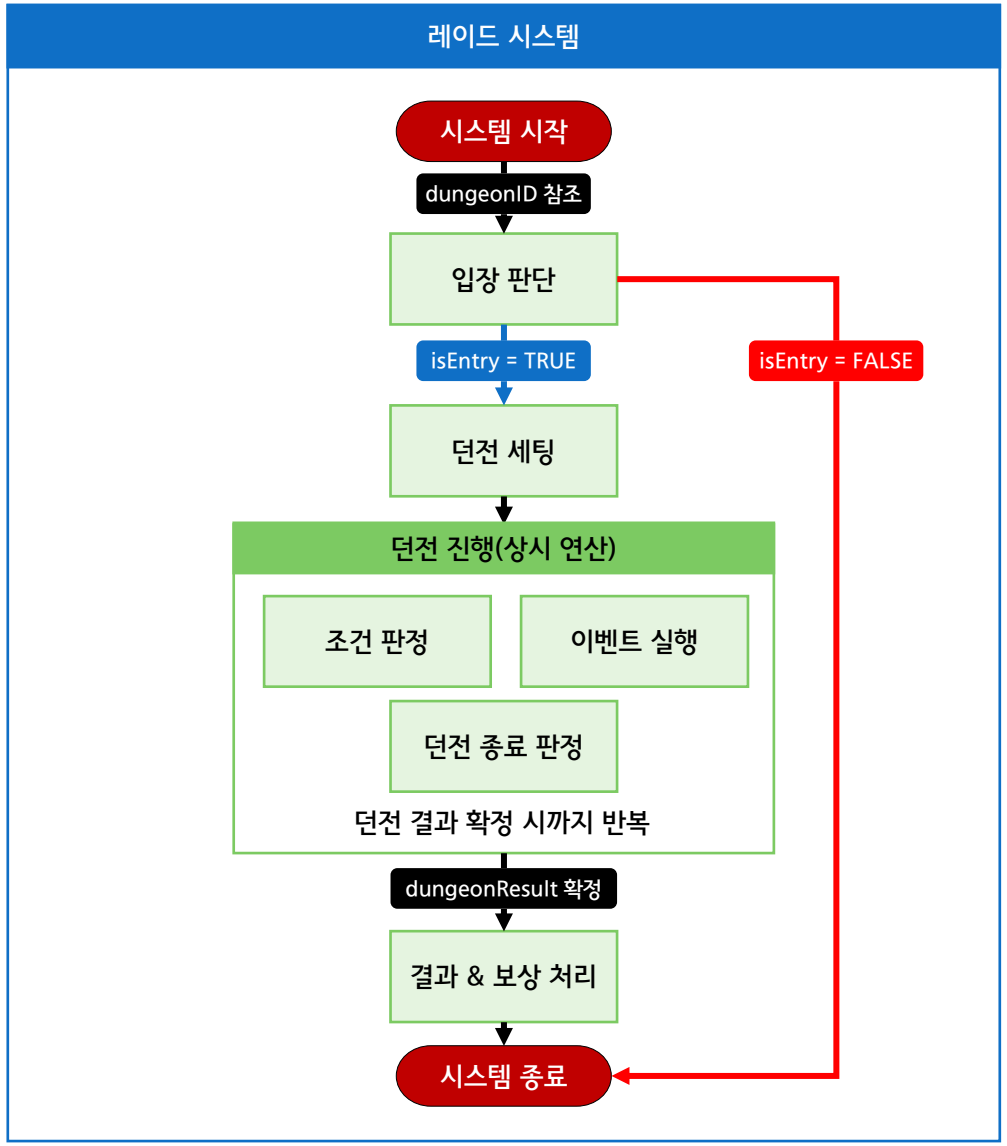
2 추가적인 이점

데이터 기반으로 던전을 관리하면 밸런스 조정이나 오류 수정 시 기존 데이터를 변경하는 방식으로 대응할 수 있다. 또한 새로운 진행 조건, 이벤트, 보상 규칙이 필요할 경우에도 기존 시스템 구조를 유지한 채 데이터를 추가하여 확장할 수 있다. 이를 통해 라이브 서비스 과정에서 발생하는 유지보수 비용을 줄이고, 지속적인 콘텐츠 업데이트에 유연하게 대응할 수 있다.



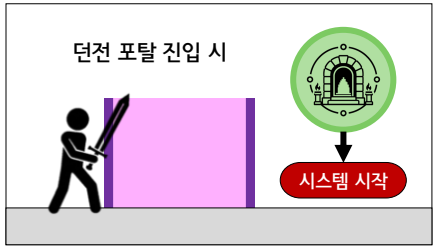
Ⅱ. 설계

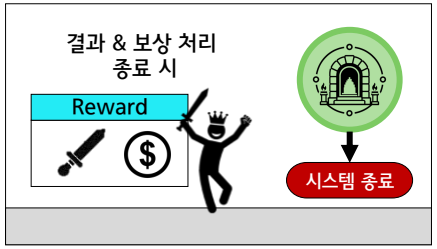
던전 시스템 구성



1 시스템 구조 개요

던전 시스템은 입장부터 종료까지의 과정을 여러 연산으로 나누어 처리하며, 시작 시 저장한 dungeonID를 시스템 종료까지 공통으로 참조한다. 각 연산은 이전 연산 종료 후 연결된 다음 연산을 실행하고, 분기 시 반환 값에 따라 실행 대상을 결정한다. 연산은 실행 방식에 따라 호출 연산과 상시 연산으로 구분한다.

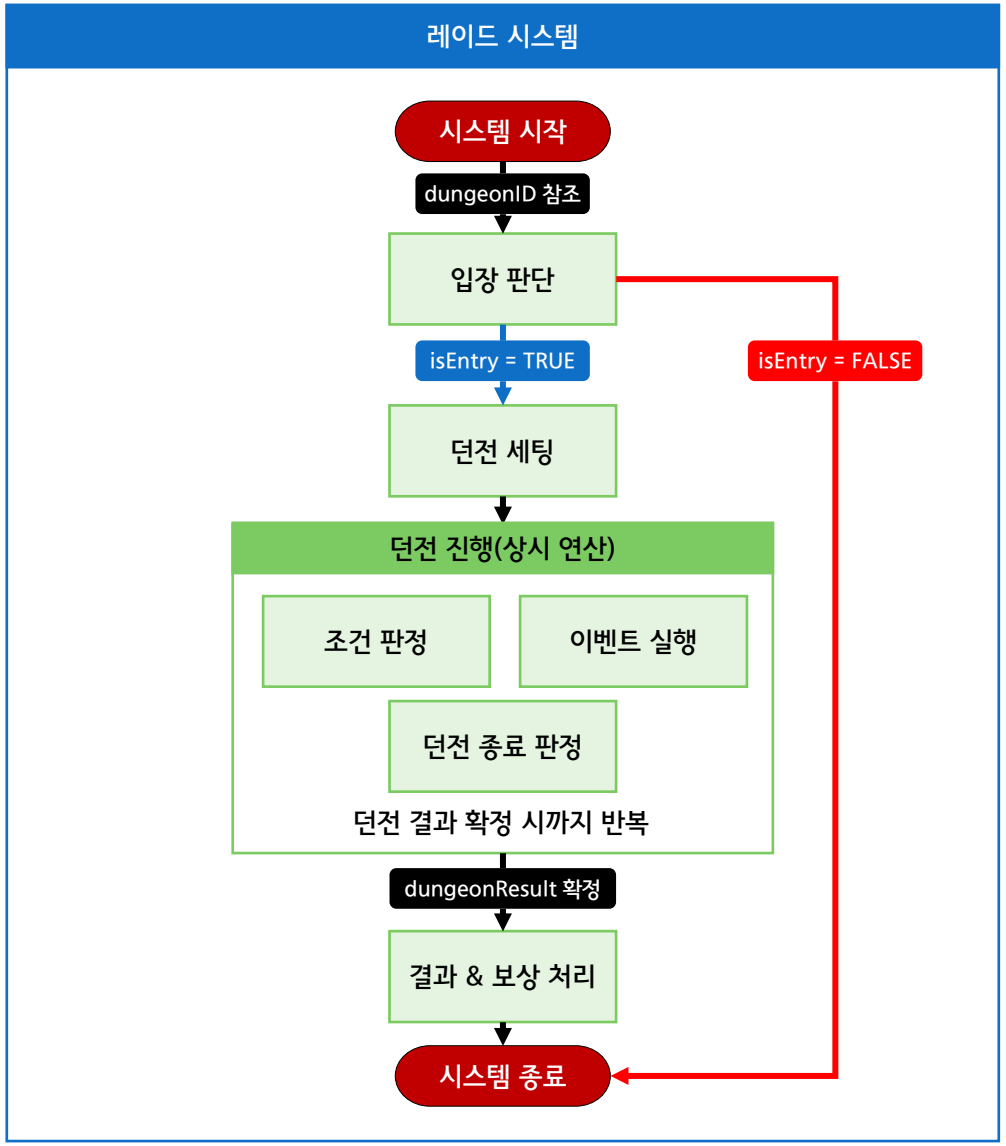




연산 구분	설명
호출 연산	이전 연산이 종료되거나 다른 연산으로부터 호출 요청을 받았을 때 실행한다. 호출된 연산은 정해진 처리를 한 차례 수행한 후 종료하며, 다시 호출되기 전까지 실행하지 않는다.
상시 연산	연산이 시작된 후 던전 결과가 확정될 때까지 종료되지 않고 반복 실행한다. 던전의 현재 상태를 지속적으로 확인하거나, 다른 연산에서 전달된 실행 요청을 처리한다.

연산 구분	연산명	설명
호출 연산	입장 판단	플레이어(또는 파티)가 던전에 입장 가능한지 판단하는 연산.
	던전 세팅	던전 내 요소들을 초기화하는 연산.
	결과 & 보상 처리	던전의 진행 결과 및 던전 종료에 따른 보상을 처리하는 연산.
상시 연산	조건 판정	진행 조건 달성 여부를 판단하는 연산.
	이벤트 실행	던전 진행 중 발생하는 이벤트를 실행하는 연산.
	던전 종료 판정	던전의 종료 여부 및 성공과 실패를 판단하는 연산.

던전 시스템 구성



2 시스템 전역 구조체 및 변수

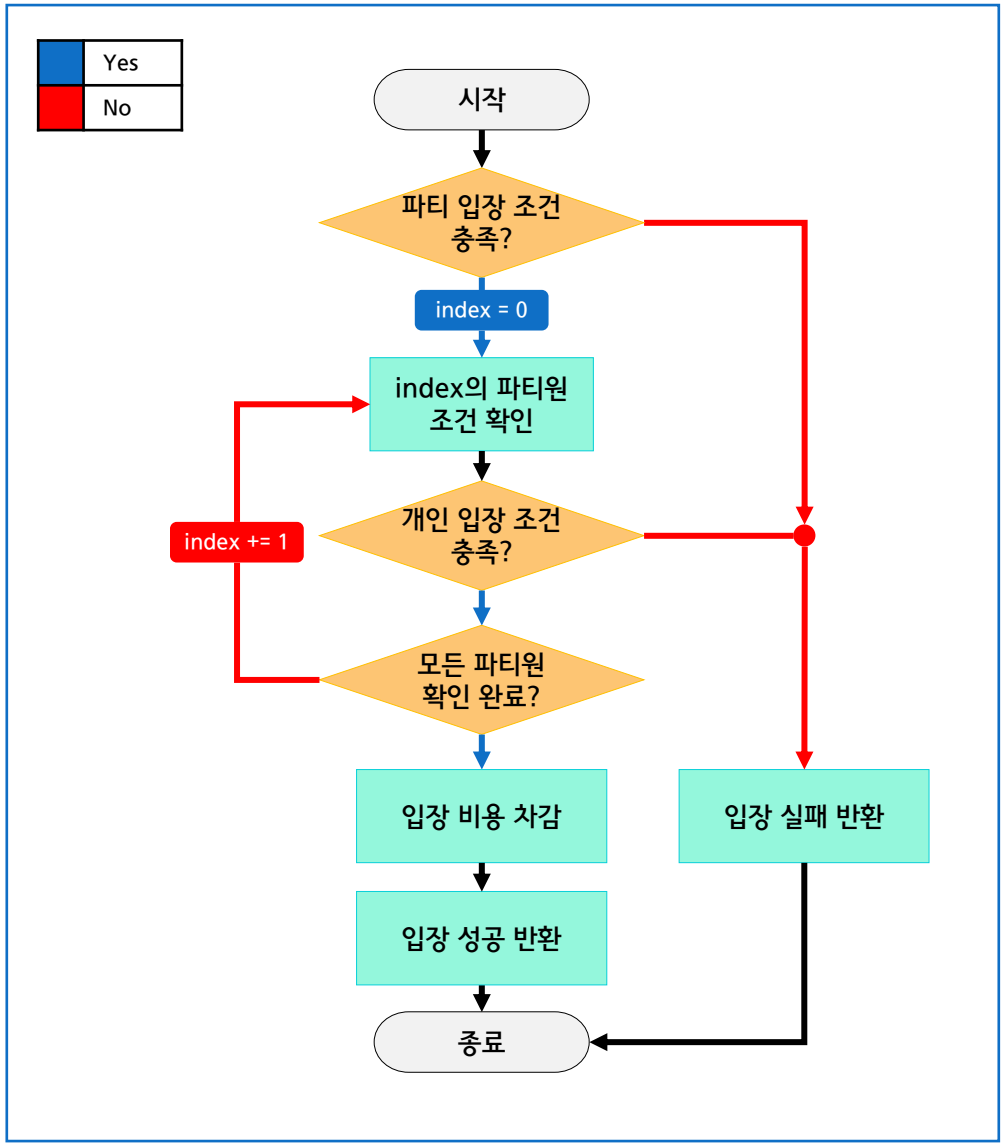
타입	구조체명	설명
시스템 구조체	ActiveCondition	진행 조건의 판정 정보와 이벤트 등록 여부를 저장한다.
	ActiveResult	종료 조건의 판정 정보와 결과 및 우선도를 저장한다.
	ActiveEvent	실행 대기 이벤트의 상태와 실행 우선도를 저장한다.
시스템 변수 (bool)	dungeonResult	종료 판정에서 확정한 던전 결과. (기본값 : NULL)
	isEntry	입장 판단 결과. (TRUE : 입장 가능 | FALSE : 입장 불가)

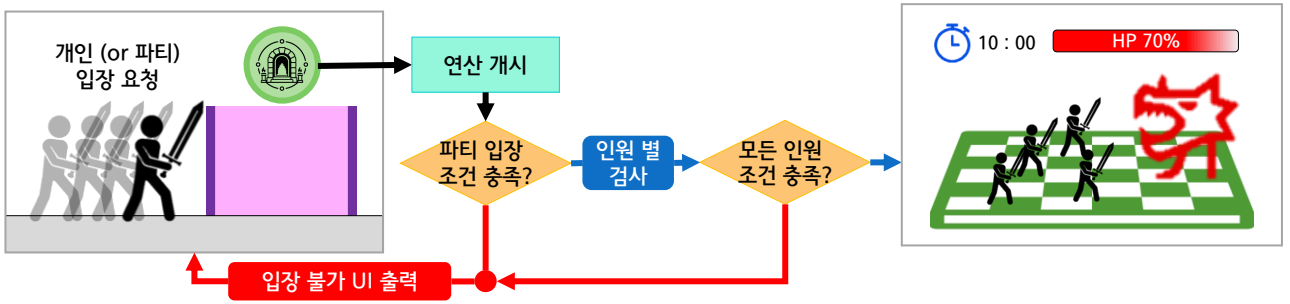
Struct : ActiveCondition			
참조 테이블	변수명	변수형	설명
ConditionList	conditionID	int	진행 조건을 구분하는 ID.
ConditionList	targetID	int	상태 값을 확인할 런타임 대상 ID.
ConditionList	targetVar	string	대상에서 확인할 변수명.
ConditionList	compareType	enum	현재 값과 기준값의 비교 방식.
ConditionList	targetValue	float	조건 달성 기준값.
ConditionList	eventID	int	조건 충족 시 등록할 이벤트 ID.
-	isMet	bool	조건 달성 여부. TRUE이면 이벤트를 다시 등록하지 않는다.

Struct : ActiveResult			
참조 테이블	변수명	변수형	설명
ConditionList	conditionID	int	종료 조건을 구분하는 ID.
ConditionList	targetID	int	상태 값을 확인할 런타임 대상 ID.
ConditionList	targetVar	string	대상에서 확인할 변수명.
ConditionList	compareType	enum	현재 값과 기준값의 비교 방식.
ConditionList	targetValue	float	종료 조건 달성 기준값.
ConditionList	resultType	bool	조건 충족 시 적용할 던전 결과.
ConditionList	priority	int	여러 종료 조건이 동시에 충족됐을 때 적용할 우선도.

Struct : ActiveEvent			
참조 테이블	변수명	변수형	설명
Event	eventID	int	실행 대기 목록에 등록된 이벤트 ID.
Event	priority	int	실행 대기 이벤트의 선택 우선도.
-	eventState	enum	이벤트의 현재 상태. (enum : Waiting, Running, Complete)

입장 판단





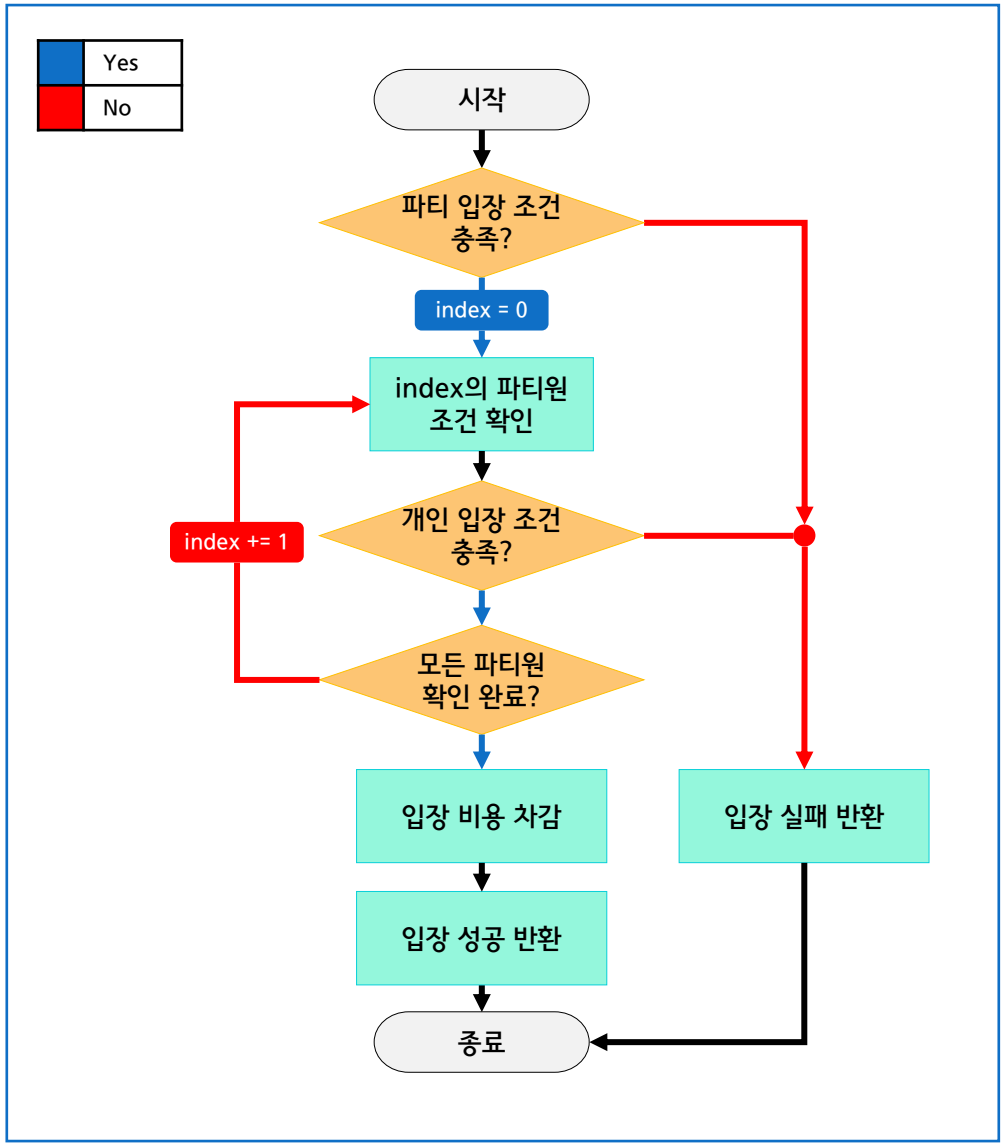
1 파티 입장 조건 확인

입장 요청을 받으면 파티 인원 수 등 파티 전체에 적용되는 조건을 먼저 확인한다.
조건을 충족하면 파티원별 조건 확인을 시작하며, 충족하지 못하면 입장 실패를 반환하고 연산을 종료한다.

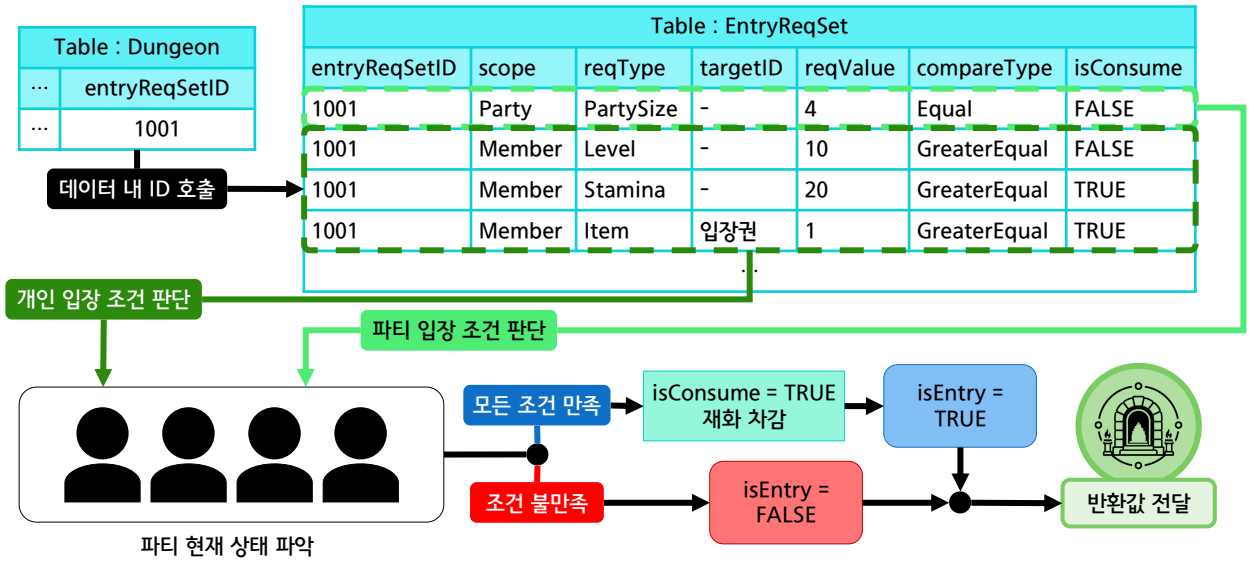
2 파티원별 입장 조건 확인

index를 기준으로 파티원을 순서대로 조회하여 개인 입장 조건을 확인한다.
한 명이라도 조건을 충족하지 못하면 입장 실패를 반환하며, 모든 파티원이 조건을 충족하면 차감 대상 비용을 처리하고 입장 성공을 반환한 뒤 연산을 종료한다.

입장 판단



※ targetID의 경우, 예시를 위해 해당 아이템 명을 기재함.
※호출 ID, 참조 ID의 경우, 예시를 위해 1~4자리로 정의함.



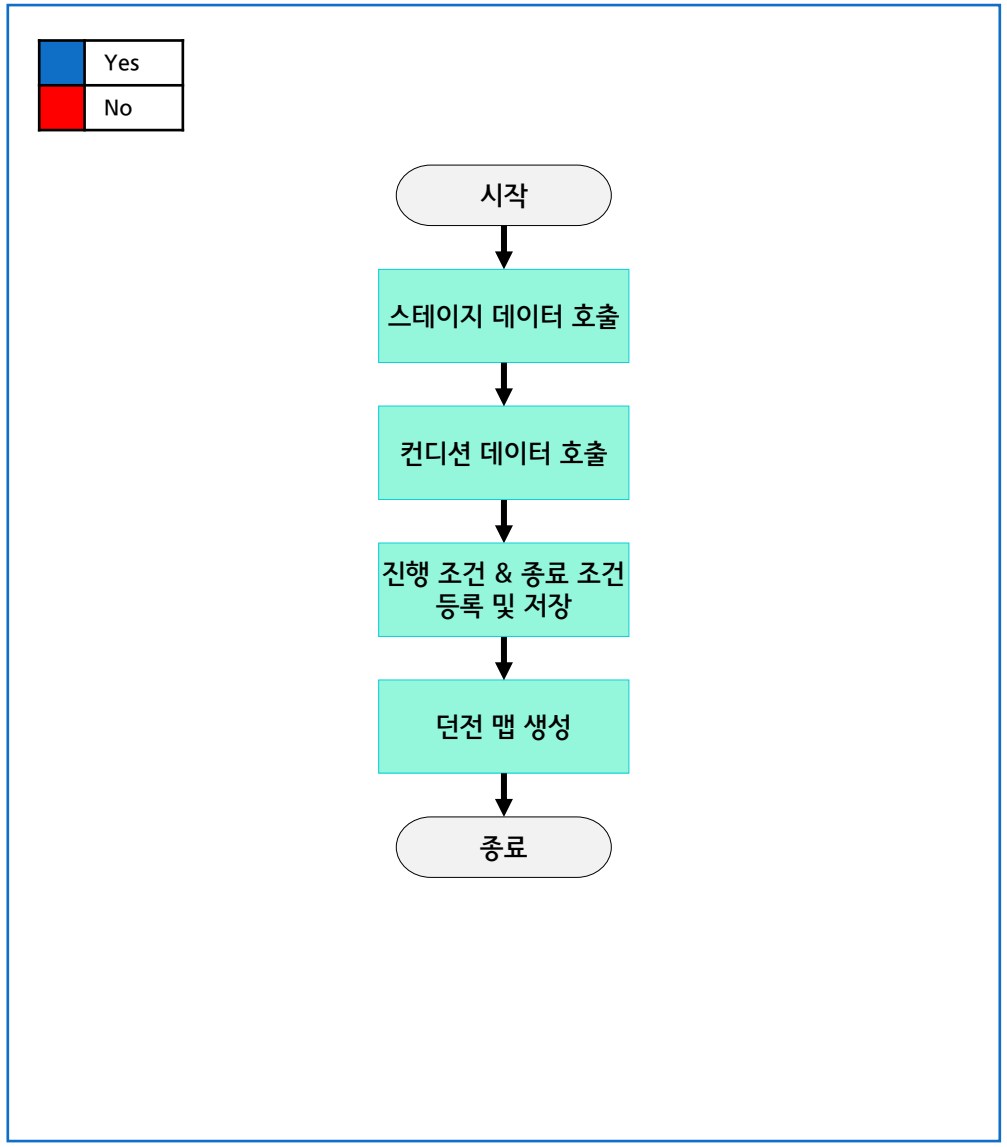
3 입장 조건 데이터 호출 및 구조화

입장 요청의 dungeonID로 Dungeon 테이블을 조회하고, entryReqSetID와 일치하는 EntryReqSet 데이터를 호출한다. 입장 판단 연산은 scope를 기준으로 파티 조건과 파티원 조건을 구분하여 각 조건 판정에 사용한다.

4 조건 데이터 활용 및 결과 반환

scope가 Party인 조건은 파티 전체에 한 번 적용하고, Member인 조건은 모든 파티원에게 반복 적용한다.
reqType과 targetID로 확인 대상을 지정하고, compareType에 따라 현재 값과 reqValue를 비교한다.
모든 조건을 충족하면 isConsume = TRUE인 항목을 차감하고 isEntry에 TRUE를 저장한다.
조건을 하나라도 충족하지 못하면 차감 없이 FALSE를 저장하여 반환한다.

던전 세팅





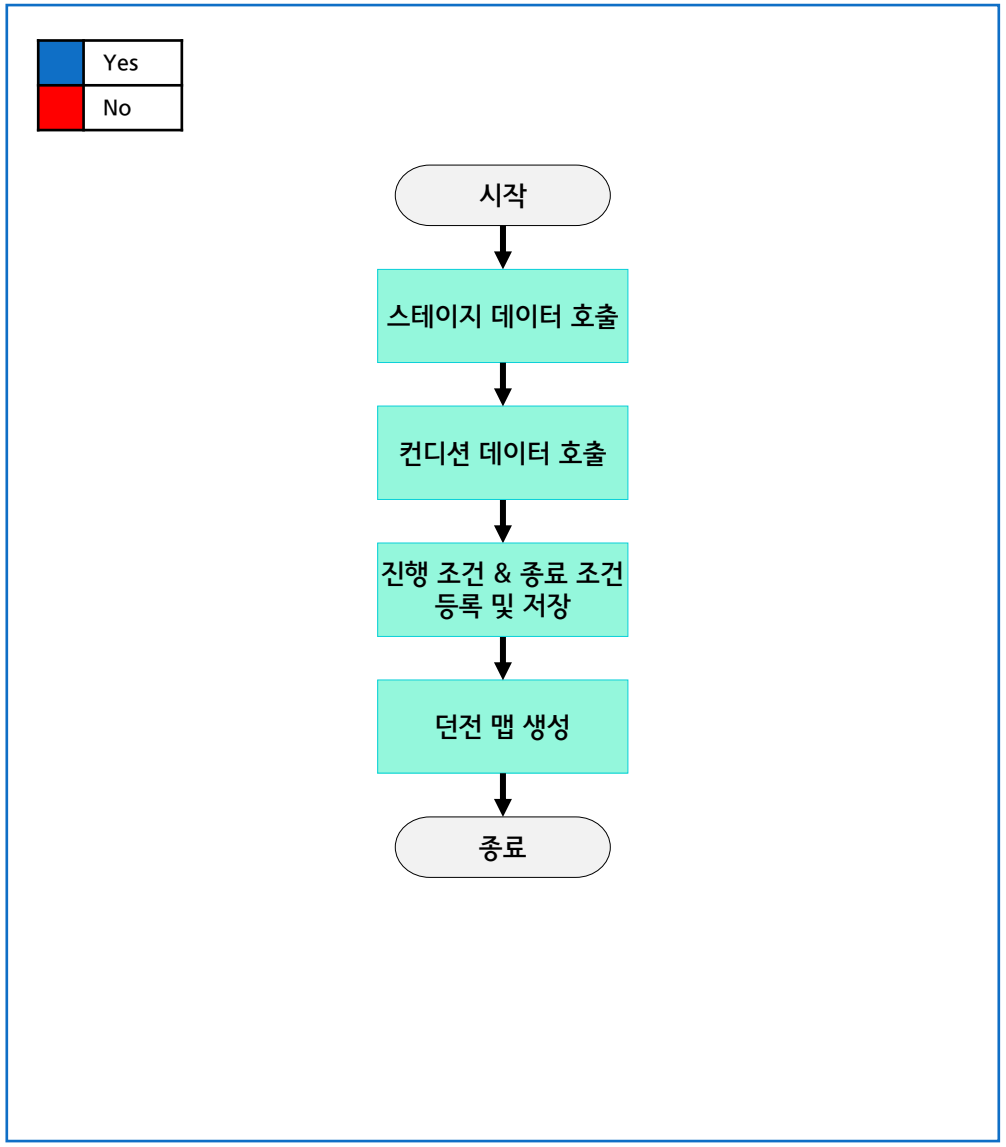
1 스테이지 및 조건 데이터 호출

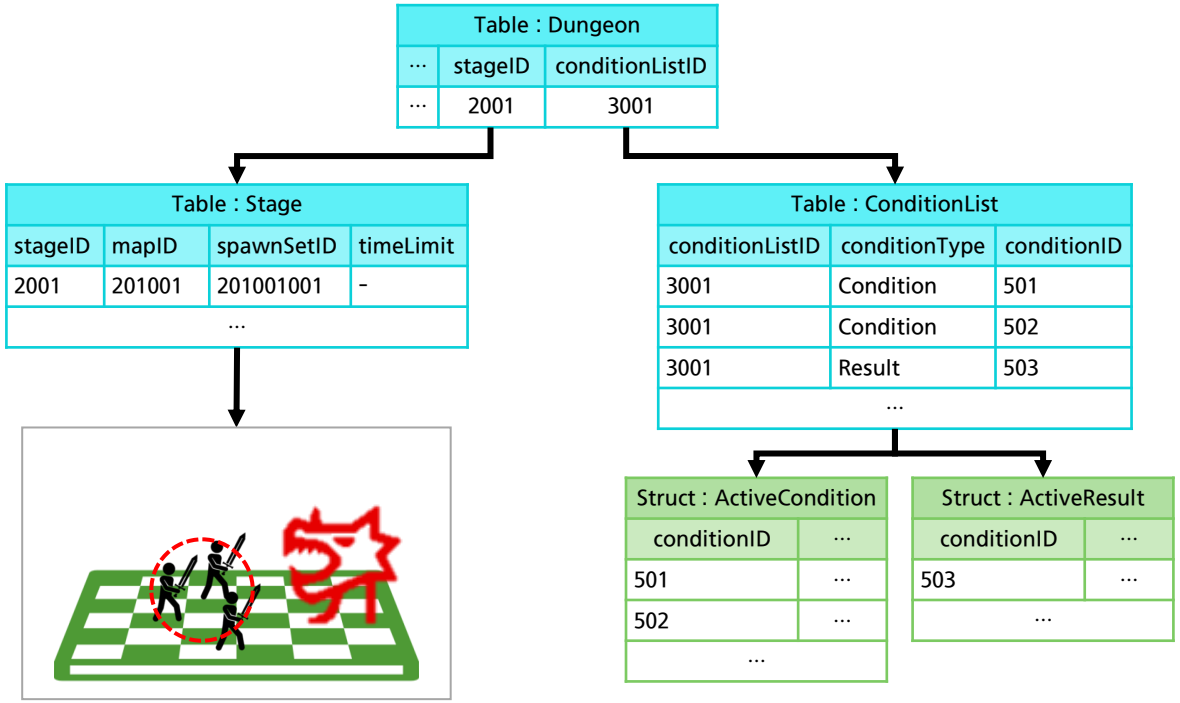
던전 세팅 연산은 현재 dungeonID를 기준으로 사용할 스테이지 데이터와 조건 목록을 순차적으로 호출한다.
던전 구성에 필요한 데이터를 모두 확인한 뒤 던전 데이터를 초기화 한다.

2 던전 데이터 초기화

호출한 스테이지 데이터에 따라 맵과 스폰 세트를 적용하고 제한 시간을 설정한다.
플레이어와 던전 내 요소를 시작 상태로 초기화하고 진행 조건 및 종료 조건을 등록한 뒤 연산을 종료한다.

던전 세팅





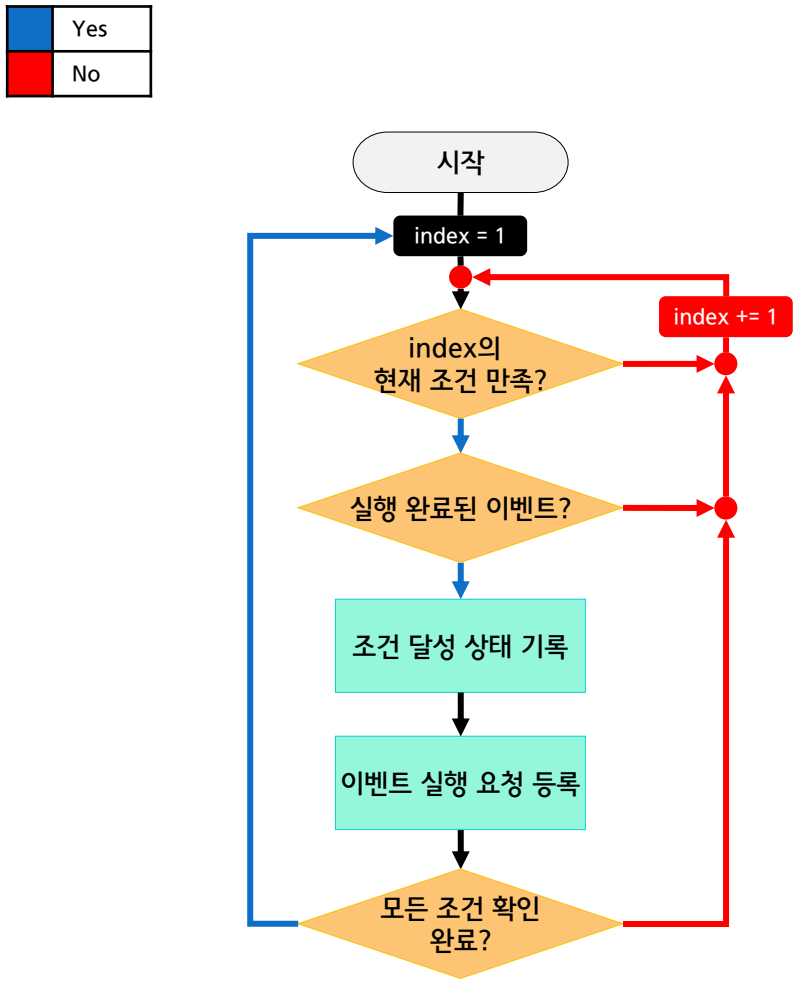
3 던전 구성 데이터 연결

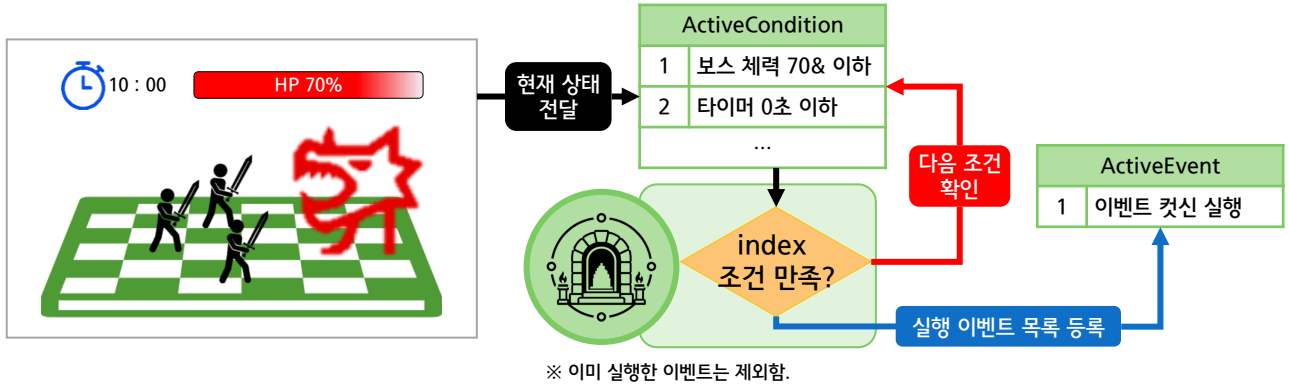
stageID와 conditionListID를 사용하여 Stage와 ConditionList 데이터를 호출한다. Stage의 mapID, spawnSetID, timeLimit은 맵 구성과 몬스터 배치 및 제한 시간 설정에 사용한다.

4 조건 데이터 분류 및 저장

ConditionList의 conditionType을 기준으로 진행 조건과 종료 조건을 구분한다.
Condition 데이터는 ActiveCondition에 저장하여 조건 판정 연산에서 사용한다.
Result 데이터는 ActiveResult에 저장하여 던전 종료 판정 연산에서 사용한다.

조건 판정





※ 이미 실행한 이벤트는 제외함.

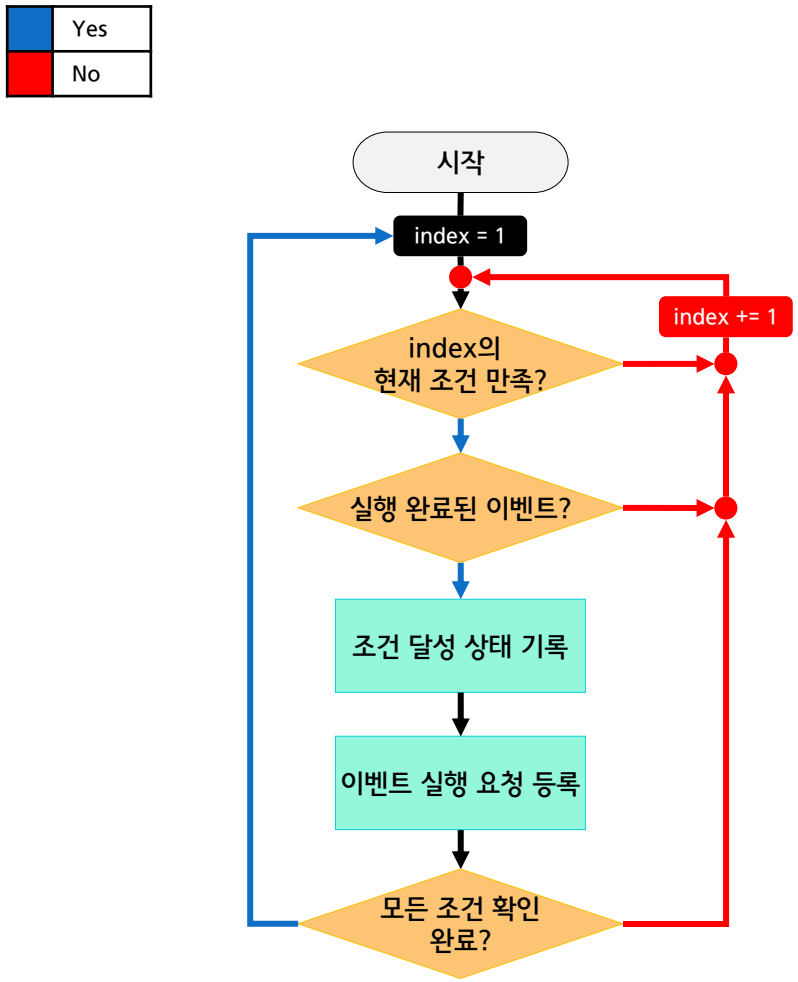
1 조건 순회 및 판정

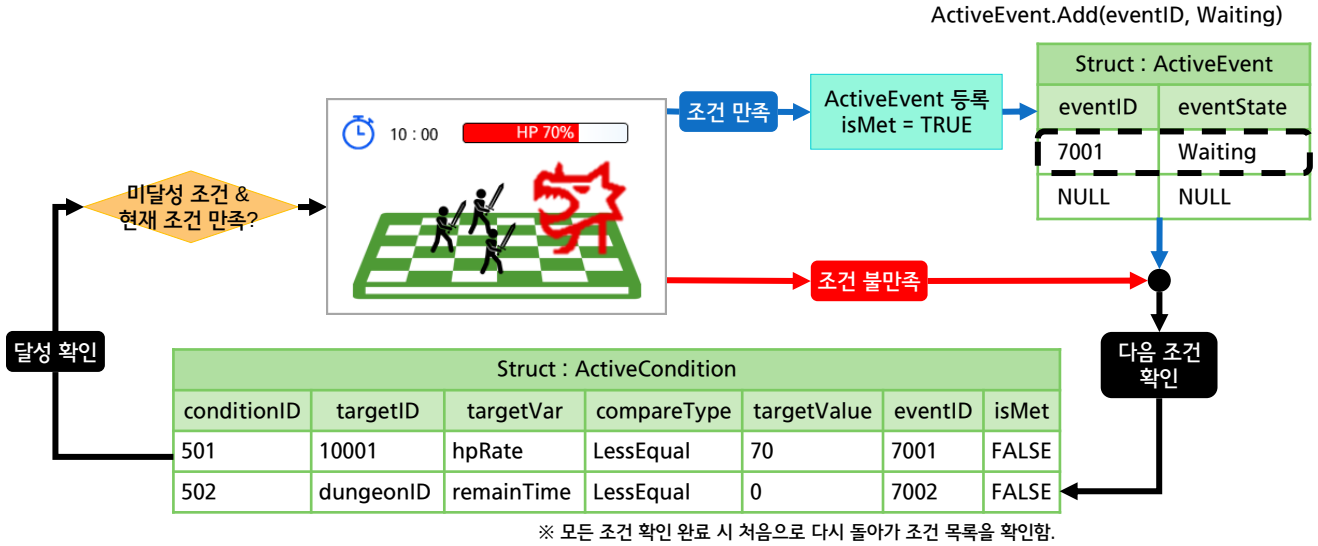
조건 판정 연산은 index = 1부터 ActiveCondition에 저장된 조건을 순서대로 확인한다. 실행하지 않은 이벤트 조건의 현재 값을 compareType과 targetValue를 기준으로 비교하며, 조건을 충족하지 못하면 다음 항목으로 이동한다.

2 달성 상태 기록 및 이벤트 실행 요청

조건을 충족하면 해당 조건을 실행 판정으로 변경하고, 연결된 이벤트를 ActiveEvent에 대기 상태로 등록한다. 동일한 이벤트가 이미 등록되었거나 실행을 완료했다면 중복 등록하지 않으며, 모든 조건을 확인하면 첫 번째 조건부터 판정을 반복한다.

조건 판정





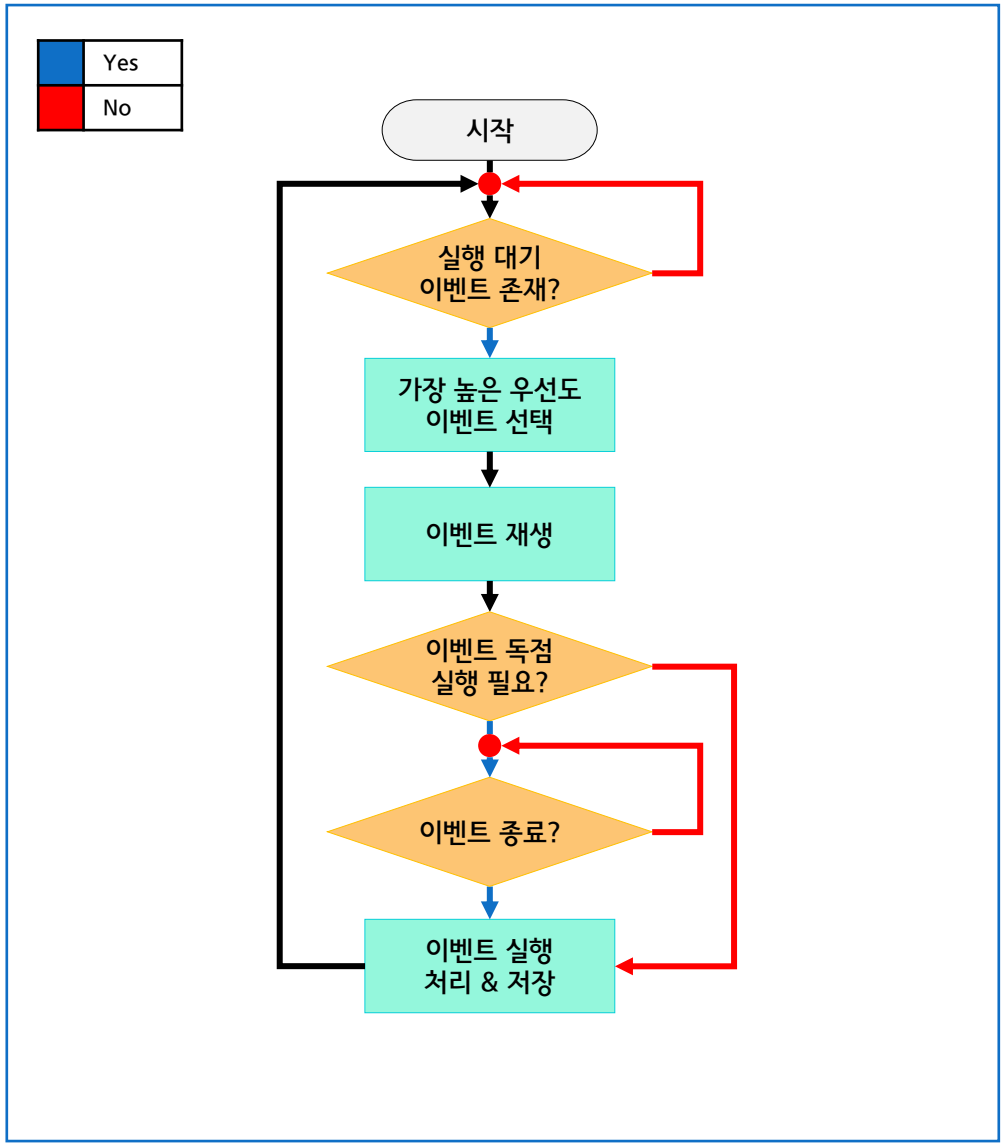
3 조건 데이터 참조 및 현재 값 확인

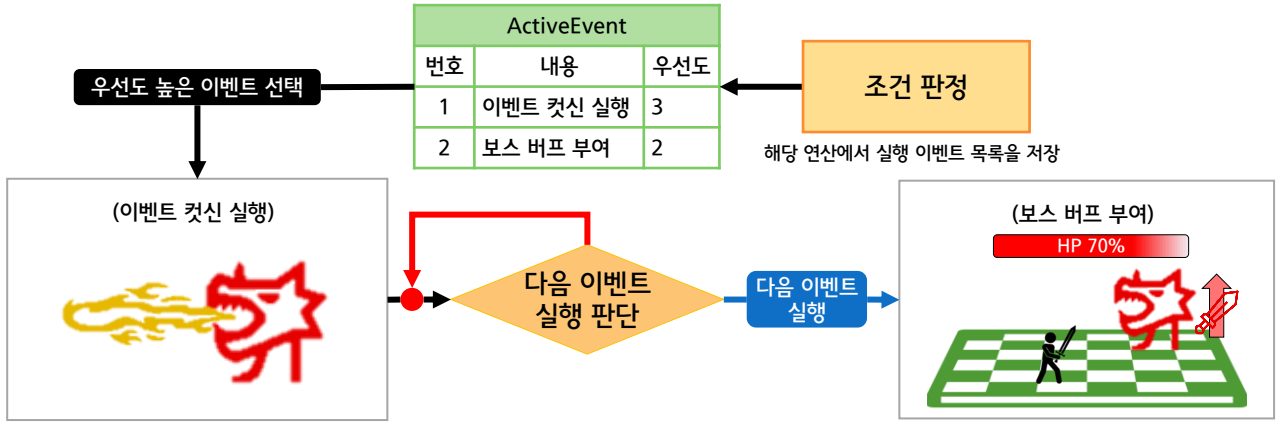
ActiveCondition에 저장된 targetID와 targetVar를 사용하여 조건 판정에 필요한 현재 값을 조회한다.
조회한 현재 값은 compareType에 따라 targetValue와 비교하여 해당 조건의 달성 여부를 판단한다.

4 조건 달성 기록 및 이벤트 등록

조건을 충족하면 해당 ActiveCondition의 isMet을 TRUE로 변경하고, 연결된 eventID를 ActiveEvent에 Waiting 상태로 등록한다.
조건을 충족하지 못하면 상태를 변경하지 않고 다음 조건을 확인하며, 모든 조건 확인 후 다시 처음부터 미달성 조건을 반복 판정한다.

이벤트 실행





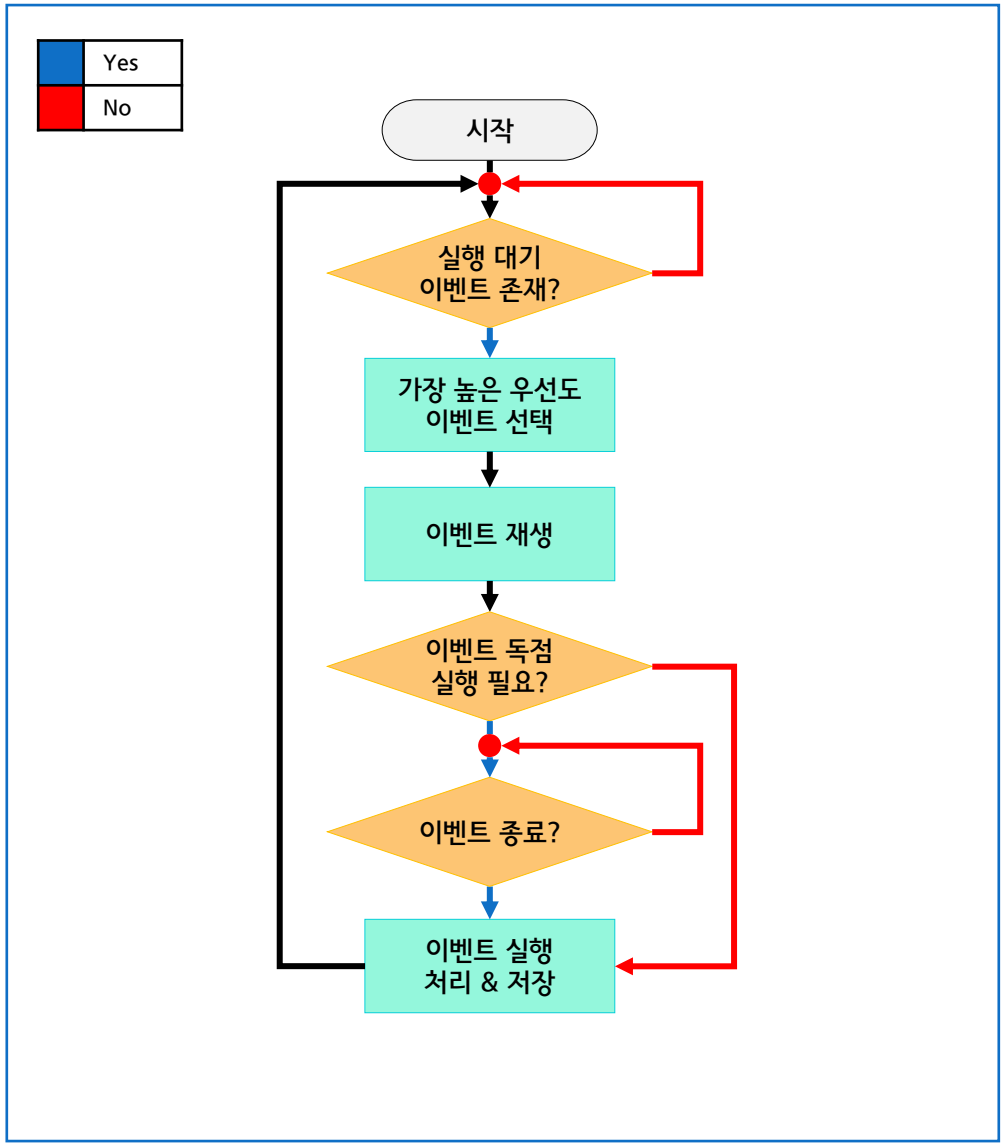
1 실행 대기 이벤트 확인

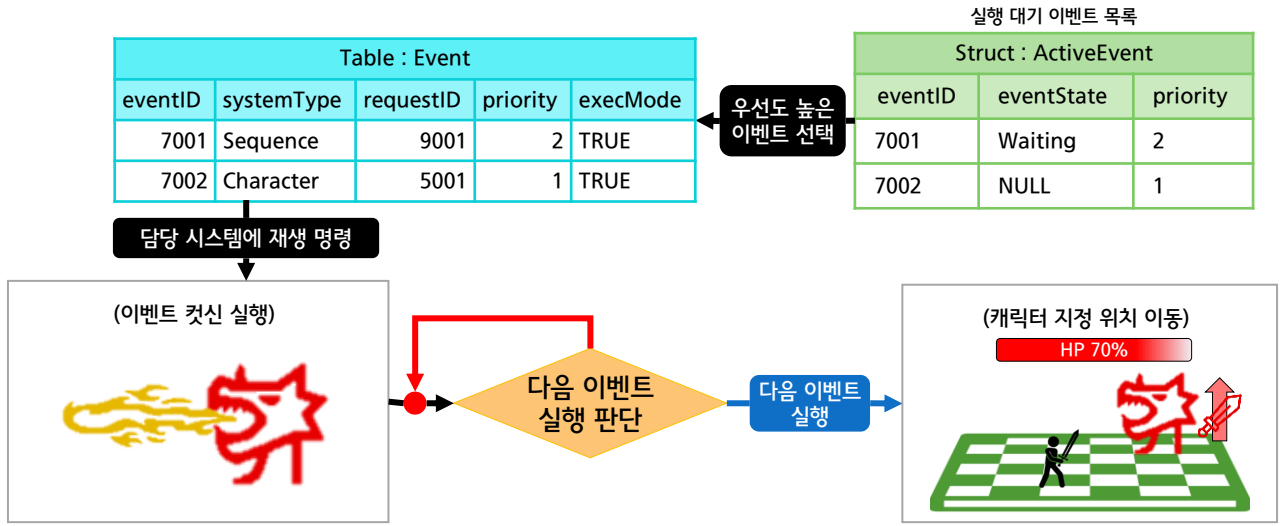
조건 판정 연산에서 ActiveEvent에 등록한 실행 대기 이벤트를 확인한다.
이벤트 실행 상태가 대기 상태인 이벤트가 존재하면 우선도를 비교하여 가장 높은 우선도의 이벤트를 현재 실행 대상으로 선택한다.

2 이벤트 실행 및 완료 처리

선택한 이벤트가 실행 중일 때, 해당 이벤트의 독점 실행 여부를 확인한다.
독점 실행이라면 다음 이벤트를 재생하지 않는다.
이벤트의 종료 여부를 지속적으로 확인하고, 실행이 완료되면 상태를 저장한 뒤 다시 실행 대기 목록을 확인하여 다음 이벤트를 처리한다.

이벤트 실행





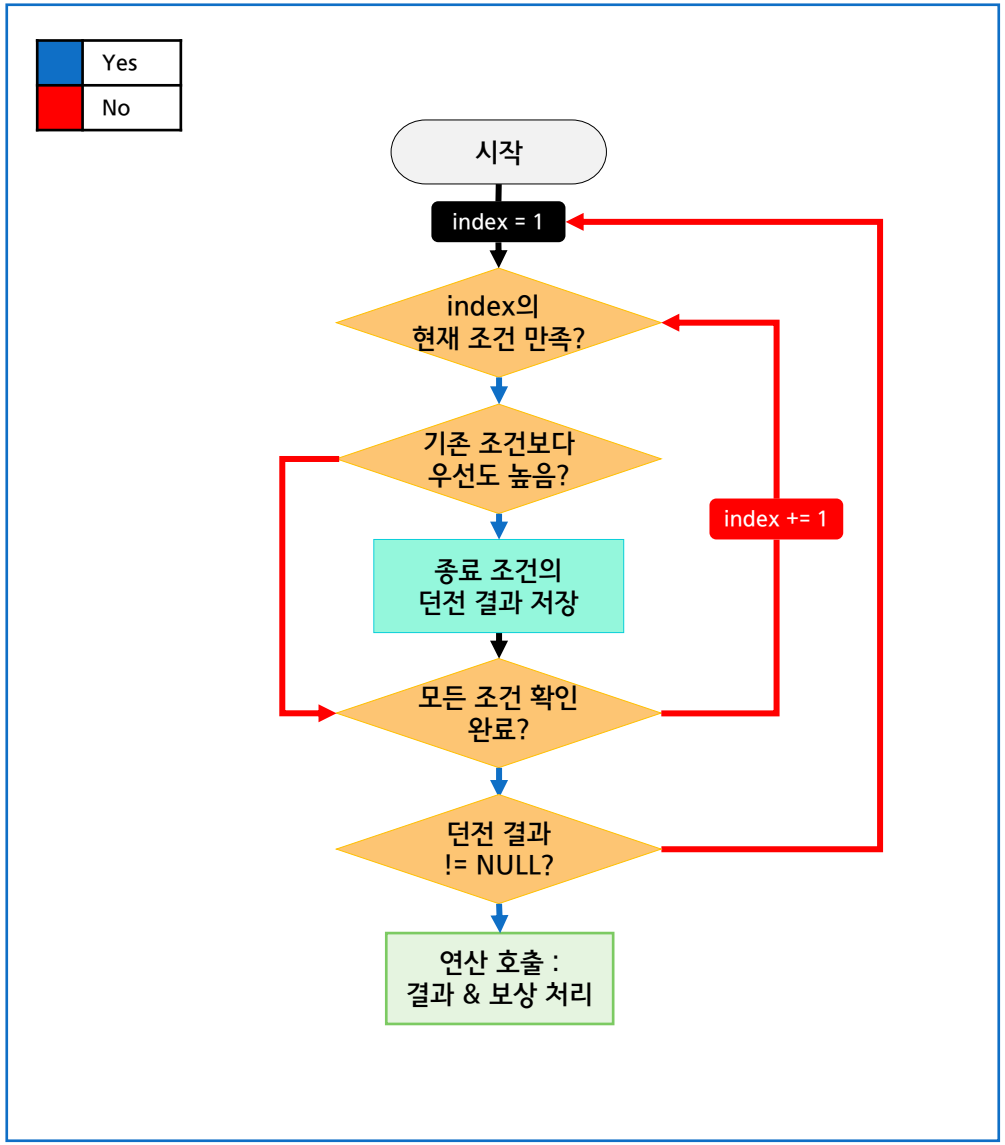
3 이벤트 데이터 조회 및 실행 요청

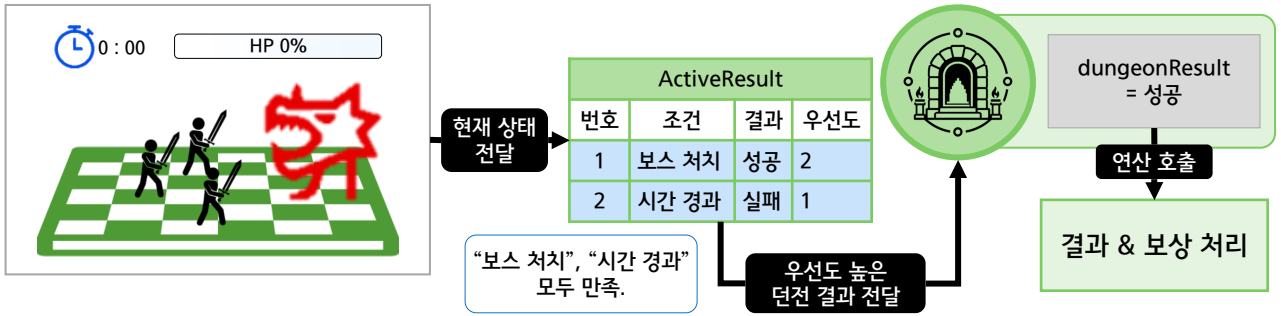
선택한 ActiveEvent의 eventID를 기준으로 Event 테이블의 실행 정보를 조회한다.
systemType으로 이벤트를 처리할 담당 시스템을 구분하고, 해당 시스템에 requestID를 전달하여 등록된 기능의 실행을 요청한다.

4 실행 상태 관리 및 다음 이벤트 선택

이벤트 실행 중에는 eventState를 통해 현재 상태를 관리하고, execMode에 따라 다음 이벤트의 실행 시점을 판단한다.
현재 이벤트 처리가 완료되면 상태를 저장하고, 남아 있는 Waiting 이벤트 중 가장 높은 priority를 가진 이벤트를 다시 선택하여 동일한 과정을 반복한다.

던전 종료 판정





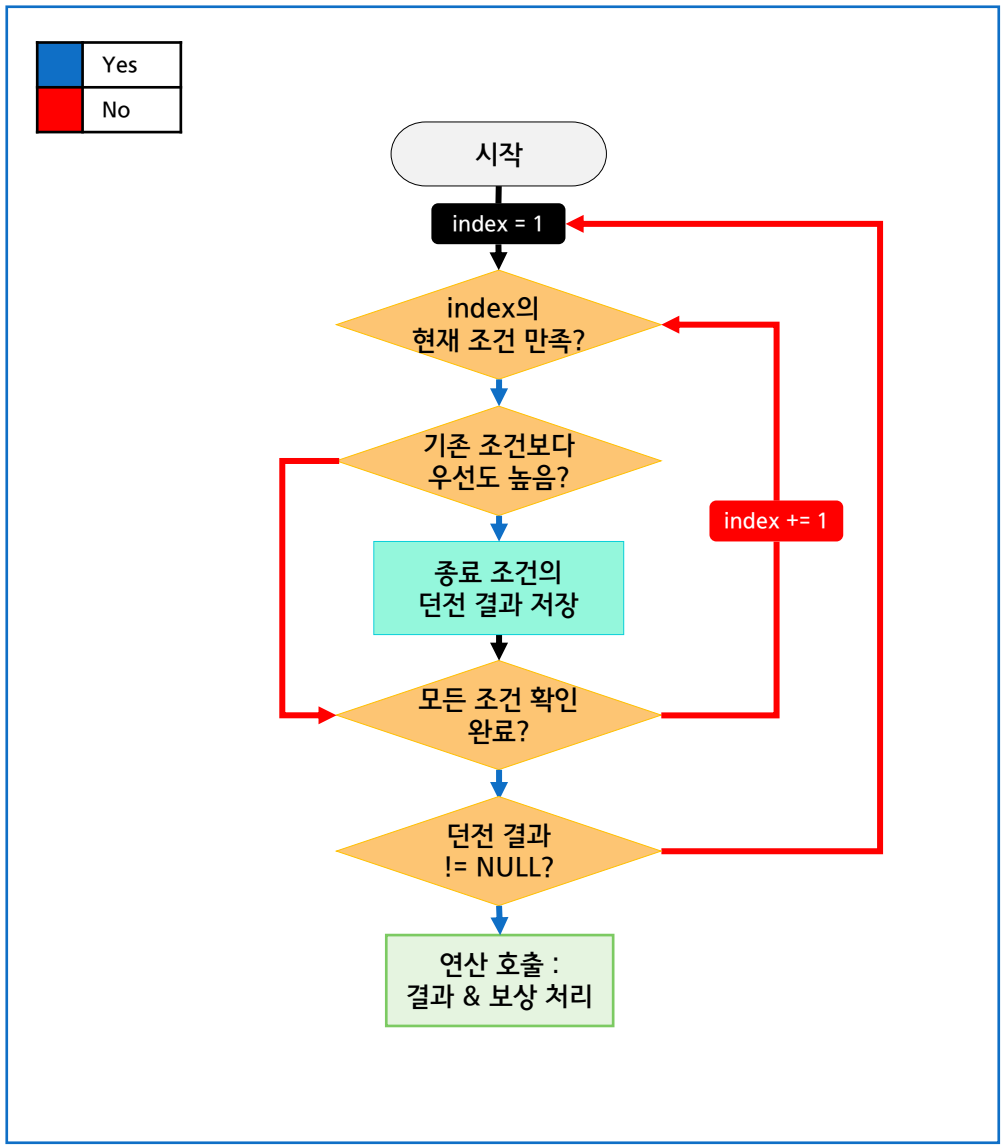
1 종료 조건 순회 및 결과 판정

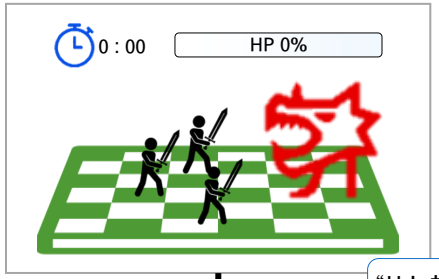
index = 1부터 ActiveResult에 저장된 종료 조건을 순서대로 확인한다.
각 조건의 현재 상태를 기준값과 비교하여 달성 여부를 판정하고, 조건을 충족한 경우 현재 저장된 결과와 우선도를 비교하여 더 높은 우선도의 결과를 dungeonResult에 저장한다.

2 종료 결과 확정 및 연산 호출

모든 종료 조건의 확인이 완료되면 저장된 던전 결과가 존재하는지 확인한다.
결과가 존재하면 가장 높은 우선도의 결과를 최종 던전 결과로 확정하고 결과 & 보상 처리 연산을 호출한다.
만족한 종료 조건이 하나도 없다면 첫 번째 조건부터 다시 판정을 반복한다.

던전 종료 판정





현재 상태 전달

"보스 처치", "시간 경과"
모두 만족.

시스템에 저장

Result
TRUE

연산 호출

결과 & 보상 처리

Struct : ActiveResult						
conditionID	targetID	targetVar	compareType	targetValue	priority	ResultType
503	10001	hpRate	LessEqual	0	2	TRUE
504	dungeonID	remainTime	LessEqual	0	1	FALSE

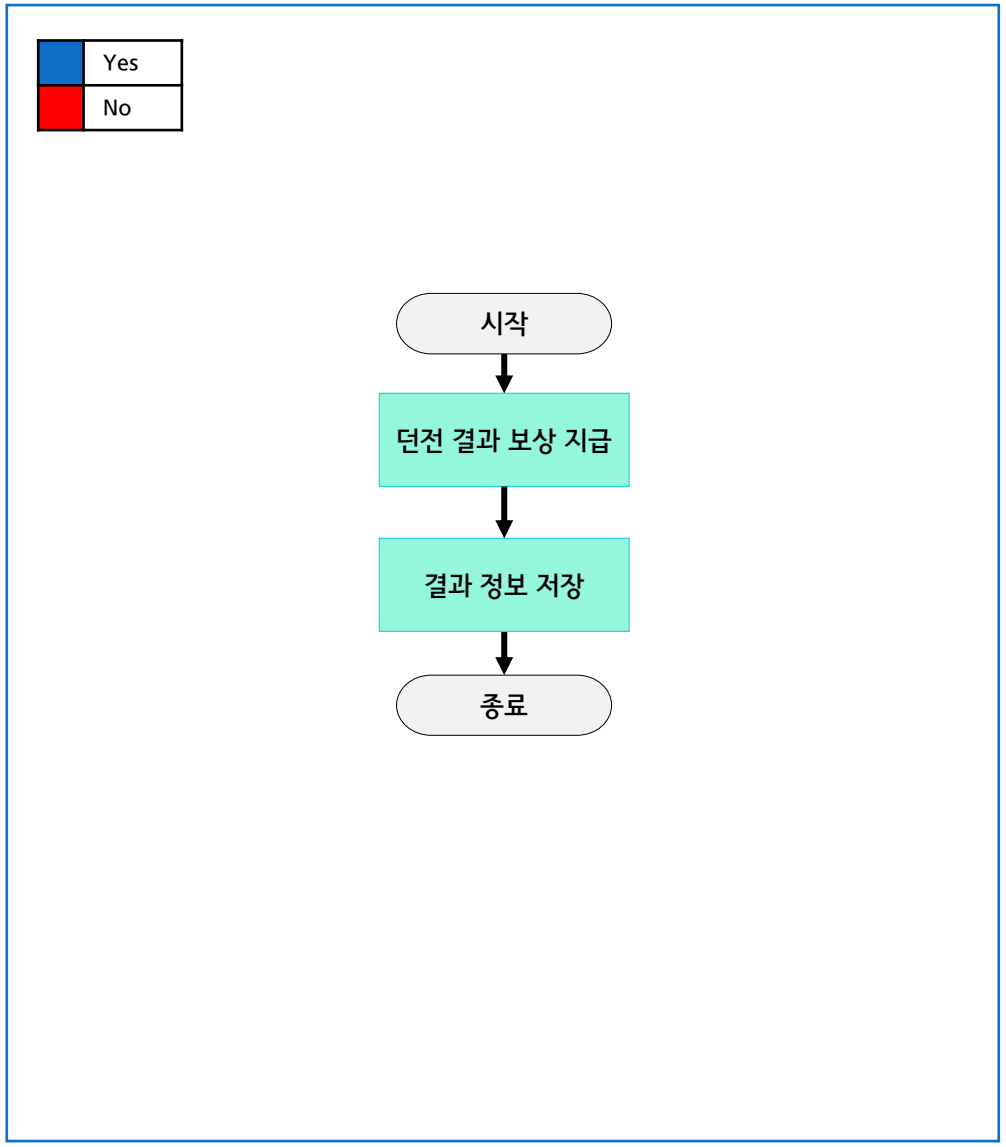
3 종료 조건 데이터 참조 및 판정

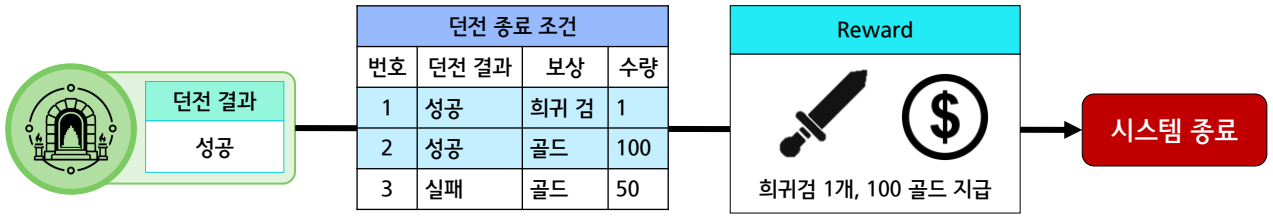
ActiveResult의 targetID와 targetVar를 사용하여 런타임에서 종료 조건의 현재 값을 조회한다.
조회한 값은 compareType에 따라 targetValue와 비교하며, 조건을 만족한 경우 해당 항목의 resultType과 priority를 결과 후보로 사용한다.

4 우선도 비교 및 던전 결과 저장

동시에 여러 종료 조건이 충족되면 각 조건의 priority를 비교하여 가장 높은 우선도의 resultType을 dungeonResult에 저장한다.
저장된 결과가 NULL이 아니면 종료 결과를 확정하고 결과 & 보상 처리 연산으로 전달한다.

결과 & 보상 처리





1 던전 결과 보상 지급

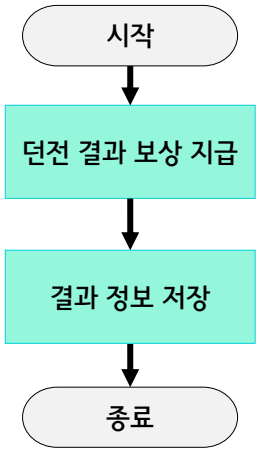
던전 종료 판정에서 확정된 던전 결과를 기준으로 지급할 보상 정보를 조회한다.
현재 결과와 일치하는 보상을 순서대로 확인하여 각 보상의 지급 대상과 수량을 적용하고 파티원에게 지급한다.

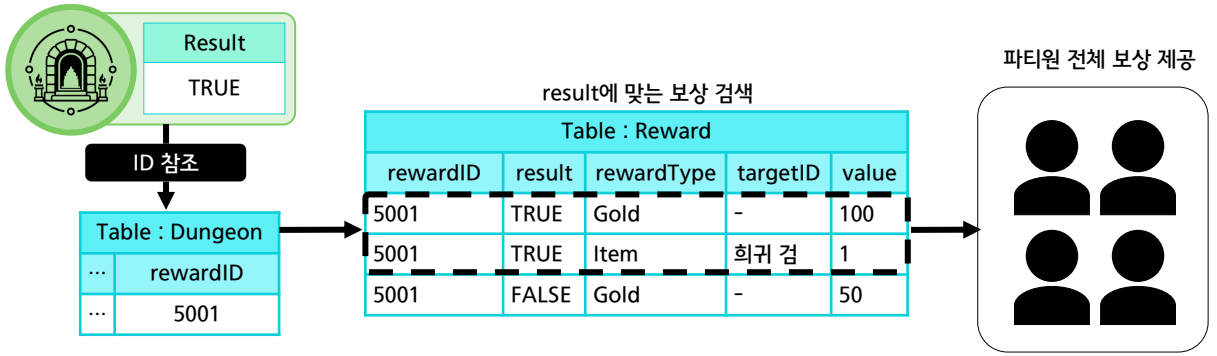
2 결과 정보 저장 및 시스템 종료

보상 지급이 완료되면 확정된 던전 결과와 보상 처리 상태를 저장한다.
필요한 결과 처리를 모두 완료한 뒤 연산을 종료하고 던전 시스템을 종료한다.

결과 & 보상 처리







3 보상 데이터 조회

현재 던전의 rewardID를 기준으로 Reward 테이블의 보상 목록을 조회한다.
확정된 던전 결과와 각 데이터의 result를 비교하여 현재 결과에 해당하는 보상만 지급 대상으로 선택한다.

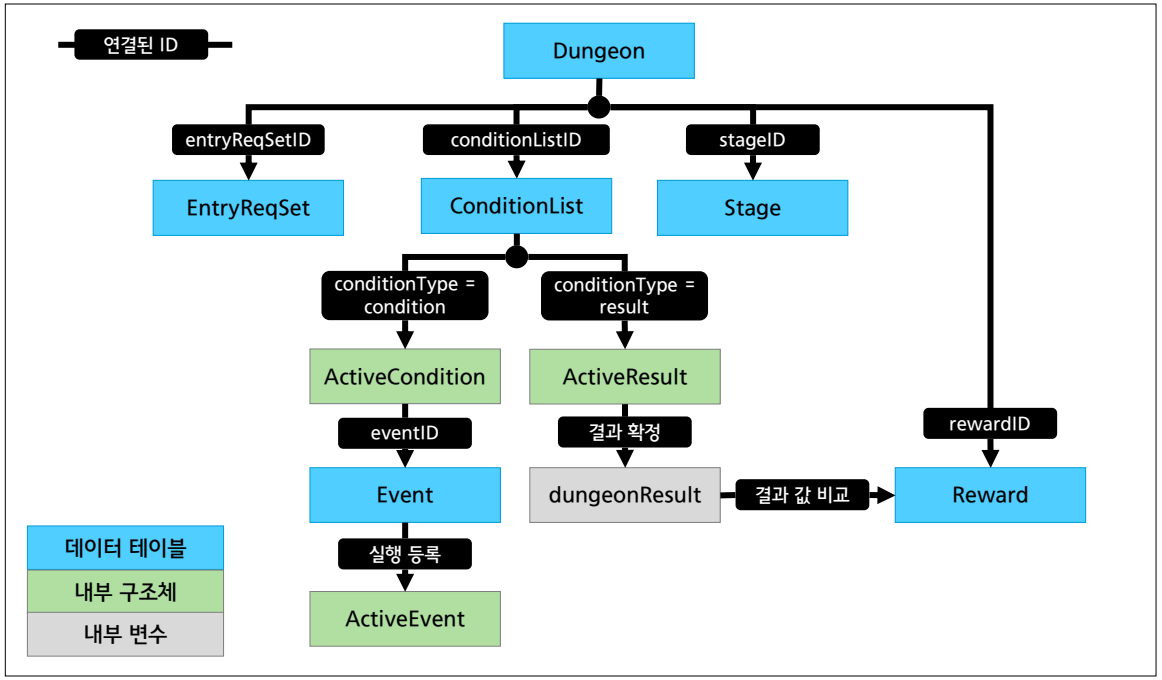
4 보상 지급 및 처리 완료

선택된 보상 데이터의 rewardType, targetID, value를 기준으로 지급할 보상의 종류와 수량을 결정한다.
동일한 결과에 연결된 모든 보상을 파티원에게 지급하고, 처리가 완료되면 던전 시스템을 종료한다.



Ⅲ. 데이터 및 변수

테이블 개요



테이블명	고유 코드	역할
Dungeon	001	던전의 최상위 기본 정보와 다른 던전 데이터의 참조 정보를 관리한다.
EntryReqSet	002	파티/파티원별 입장 조건과 입장 시 차감할 비용 조건을 관리한다.
Stage	003	던전에서 사용할 맵, 스폰 구성, 제한 시간 등 스테이지 환경 정보를 관리한다.
ConditionList	004	던전 진행 조건과 종료 조건을 하나의 조건 목록으로 관리한다.
Event	005	조건 충족 시 실행할 이벤트와 담당 시스템에 전달할 실행 정보를 관리한다.
Reward	006	던전 결과에 따라 지급할 보상 정보를 관리한다.

1 데이터 테이블 개요

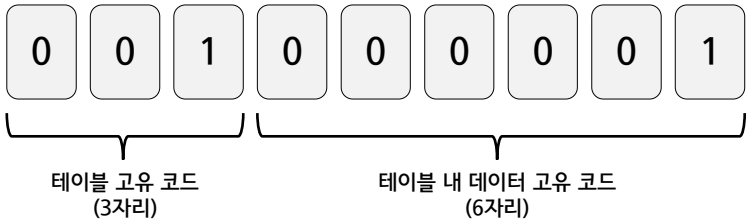
Dungeon을 중심으로 던전 진행에 필요한 데이터를 역할별 테이블로 분리하여 관리한다.
각 테이블은 ID를 통해 연결하며, 필요한 데이터의 조합으로 던전별 구성을 정의한다.

2 참조 ID & 호출 ID

1. 참조 ID
   해당 테이블의 데이터 또는 데이터 그룹을 식별하며, 다른 테이블에서 참조할 때 기준이 되는 ID.
2. 호출 ID
   다른 테이블의 참조 ID를 지정하여 필요한 데이터를 호출하는 ID.

3 ID작성 규칙

1. ID는 9자리의 양의 정수로 정의한다.
2. 앞 3자리는 데이터 테이블을 구분하는 고유 코드로 사용한다.
3. 뒤 6자리는 해당 테이블 내 데이터를 구분하는 일련번호로 사용한다.



테이블 범례

Table : Dungeon					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
dungeonID	int	-	던전을 구분하는 고유 ID	참조 ID	
dungeonName	string	-	던전 이름	-	-
dungeonType	Enum	Instance	던전 유형	-	-
entryReqSetID	int	-	사용할 입장 조건 데이터를 호출하는 ID	호출 ID	EntryReqSet
stageID	int	-	사용할 스테이지 데이터를 호출하는 ID	호출 ID	Stage
conditionListID	int	-	사용할 조건 목록을 호출하는 ID	호출 ID	ConditionList
rewardID	int	-	사용할 보상 정보를 호출하는 ID	호출 ID	Reward
Table : EntryReqSet					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
entryReqSetID	int	-	동일 ID의 조건을 하나의 입장 조건 세트로 구성하는 ID	참조 ID	Dungeon
scope	Enum	Party	조건 적용 범위	-	-
reqType	Enum	None	입장 조건 확인에 사용할 요구 항목 유형	-	-
targetID	int	0	조건 판정 대상 데이터를 호출하는 ID	호출 ID	Item / character 등
reqValue	float	0	입장 조건 충족 기준값	-	-
compareType	Enum	Equal	현재 값과 기준값의 비교 방식	-	-
isConsume	bool	FALSE	조건 충족 시 차감 여부 (TRUE : 개수 및 횟수 차감 / FALSE : 차감 없음)	-	-
Table : Stage					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
stageID	int	-	Dungeon에서 호출되는 스테이지 ID	참조 ID	Dungeon
mapID	int	-	사용할 맵 데이터를 호출하는 ID	호출 ID	Map
spawnSetID	int	-	사용할 스폰 세트 데이터를 호출하는 ID	호출 ID	SpawnSet
timeLimit	float	0	던전 제한 시간. 0은 제한 없음	-	-

테이블 범례

Table : ConditionList					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
conditionListID	int	-	동일 ID의 조건을 하나의 조건 목록으로 구성하는 ID	참조 ID	Dungeon
conditionID	int	-	각 조건 데이터를 구분하는 고유 ID	-	-
groupID	int	0	둘 이상의 조건을 하나의 논리식으로 묶는 그룹 번호	-	-
logicType	bool	TRUE	동일 그룹 내 조건의 논리 연산 방식(TRUE : AND / FALSE : OR)	-	-
conditionType	Enum	Condition	진행 조건 / 종료 조건 구분	-	-
targetID	int	0	상태 값을 확인할 런타임 대상 ID	호출 ID	Monster / Object / System 등
targetVar	string	-	대상에서 확인할 변수명	-	-
compareType	Enum	Equal	현재 값과 기준값의 비교 방식	-	-
targetValue	float	0	조건 달성 기준값	-	-
eventID	int	0	진행 조건 충족 시 실행할 이벤트 ID	호출 ID	Event
resultType	bool	TRUE	종료 조건 충족 시 적용할 던전 결과 (TRUE : 성공 / FALSE : 실패)	-	-
priority	int	0	여러 종료 조건이 동시에 충족됐을 때 적용할 우선도	-	-
Table : Event					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
eventID	int	-	조건 충족 시 호출되는 이벤트 ID	참조 ID	ConditionList
systemType	string	-	이벤트를 처리할 담당 시스템 유형	-	-
requestID	int	-	담당 시스템에 전달할 실행 요청 ID	호출 ID	Sequence / Character / UI 등
priority	int	0	실행 대기 이벤트 간 선택 우선도	-	-
execMode	bool	TRUE	이벤트 실행 방식 (TRUE : 독점 실행 / FALSE : 병행 실행)	-	-

테이블 범례

Table : Reward					
컬럼명	자료형	기본값	설명	ID 구분	참조 테이블
rewardID	int	-	Dungeon에서 호출되는 보상 정보 ID	참조 ID	Dungeon
result	bool	TRUE	보상을 지급할 던전 결과 (TRUE : 성공 / FALSE : 실패)	-	-
rewardType	Enum	Item	지급할 보상 유형	-	-
targetID	int	0	지급할 보상 데이터를 호출하는 ID	호출 ID	Item / Currency / EXP 등
value	int	0	지급할 보상 수량 또는 값	-	-

Table : Enum					
Enum명	값	설명	Enum명	값	설명
DungeonType	Instance	일반 인스턴스 던전	CompareType	Equal	현재 값 = 기준값
	Raid	레이드 던전		NotEqual	현재 값 ≠ 기준값
	Maze	메이즈 던전		Greater	현재 값 > 기준값
Scope	Party	파티 전체에 조건 적용		GreaterEqual	현재 값 ≥ 기준값
	Member	각 파티원에게 조건 적용		Less	현재 값 < 기준값
ReqType	PartySize	파티 인원 수 확인		LessEqual	현재 값 ≤ 기준값
	Level	캐릭터 레벨 확인	RewardType	Item	아이템 보상
	Stamina	행동력 보유량 확인		Gold	골드 보상
	Item	아이템 보유량 확인		EXP	경험치 보상
ConditionType	Condition	던전 진행 조건		Currency	특수 재화 보상
	Result	던전 종료 조건			